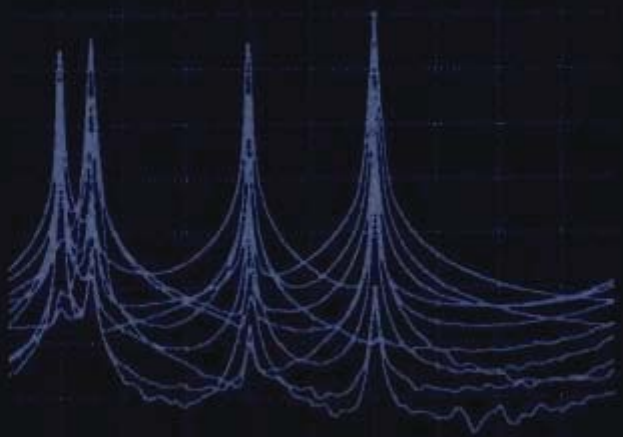
Copyrighted Material

MONSON H. HAYES

STATISTICAL DIGITAL SIGNAL PROCESSING AND MODELING



Copyrighted Material

8.8 Summary 473
8.9 Problems 477

9 ADAPTIVE FILTERING 493
9.1 Introduction 493
9.2 FIR Adaptive Filters 497
9.2.1 The Steepest Descent Adaptive Filter 499
9.2.2 The LMS Algorithm 505
9.2.3 Convergence of the LMS Algorithm 506
9.2.4 Normalized LMS 514
9.2.5 Application: Noise Cancellation 516
9.2.6 Other LMS-Based Adaptive Filters 521
9.2.7 Gradient Adaptive Lattice Filter 526
9.2.8 Joint Process Estimator 528
9.2.9 Application: Channel Equalization 530
9.3 Adaptive Recursive Filters 534
9.4 Recursive Least Squares 541
9.4.1 Exponentially Weighted RLS 541
9.4.2 Sliding Window RLS 548
9.4.3 Summary 551
9.5 Problems 554

Appendix USE OF MATLAB PROGRAMS 571
A.1 Introduction 571
A.2 General Information 572
A.3 Random Processes 574
A.4 Signal Modeling 574
A.5 Levinson Recursion 578
A.6 Lattice Filters 582
A.7 Optimum Filters 583
A.8 Spectrum Estimation 584
A.9 Adaptive Filtering 592

Table of Symbols 595

Index 599

9.4 RECURSIVE LEAST SQUARES

In each of the adaptive filtering methods discussed so far, we have considered gradient descent algorithms for the minimization of the *mean-square error*

$$\xi(n) = E\{|e(n)|^2\}$$

The difficulty with these methods is that they all require knowledge of the autocorrelation of the input process, $E\{x(n)x^*(n-k)\}$, and the cross-correlation between the input and the desired output, $E\{d(n)x^*(n-k)\}$. When this statistical information is unknown, we have been forced to estimate these statistics from the data. In the LMS adaptive filter, for example, these ensemble averages are estimated using instantaneous values,

$$\hat{E}\{e(n)x^*(n-k)\} = e(n)x^*(n-k) \tag{9.88}$$

Although this approach may be adequate in some applications, in others this gradient estimate may not provide a sufficiently rapid rate of convergence or a sufficiently small excess mean-square error. An alternative, therefore, is to consider error measures that do not include expectations and that may be computed directly from the data. For example, a least squares error

$$\mathcal{E}(n) = \sum_{i=0}^{n} |e(i)|^2$$

requires no statistical information about $x(n)$ or $d(n)$, and may be evaluated directly from $x(n)$ and $d(n)$. There is an important philosophical difference, however, between minimizing the least squares error and the mean-square error. Minimizing the mean-square error produces the same set of filter coefficients for all sequences that have the same statistics. Therefore, the coefficients do not depend on the incoming data, only on their ensemble average. With the least squares error, on the other hand, we are minimizing a squared error that depends explicitly on the specific values of $x(n)$ and $d(n)$. Consequently, for different signals we get different filters. As a result, the filter coefficients that minimize the least squares error will be optimal for the given data rather than statistically optimal over a particular class of process. In other words, different realizations of $x(n)$ and $d(n)$ will lead to different solutions, even if the statistics of these sequences are the same. In this section, we will look at the filters that are derived by minimizing a weighted least squares error, and derive an efficient algorithm for performing this minimization known as *recursive least squares*.

9.4.1 Exponentially Weighted RLS

Let us reconsider the design of an FIR adaptive Wiener filter and find the filter coefficients

$$\mathbf{w}_n = \left[w_n(0),\ w_n(1),\ \ldots,\ w_n(p)\right]^T$$

that minimize, at time n, the weighted least squares error

$$\mathcal{E}(n) = \sum_{i=0}^{n} \lambda^{n-i} |e(i)|^2 \tag{9.89}$$

where $0 < \lambda \leq 1$ is an exponential weighting (forgetting) factor and

$$e(i) = d(i) - y(i) = d(i) - \mathbf{w}_n^T \mathbf{x}(i) \tag{9.90}$$

Note that $e(i)$ is the difference between the desired signal $d(i)$ and the filtered output at time i, using the *latest* set of filter coefficients, $w_n(k)$. Thus, in minimizing $\mathcal{E}(n)$ it is assumed that

the weights $\mathbf{w}_n$ are held constant over the entire observation interval $[0, n]$. To find the coefficients that minimize $\mathcal{E}(n)$ we proceed exactly as we have done many times before by setting the derivative of $\mathcal{E}(n)$ with respect to $w_n^*(k)$ equal to zero for $k = 0, 1, \ldots, p$. Thus, we have

$$\frac{\partial \mathcal{E}(n)}{\partial w_n^*(k)} = \sum_{i=0}^{n} \lambda^{n-i} e(i) \frac{\partial e^*(i)}{\partial w_n^*(k)} = -\sum_{i=0}^{n} \lambda^{n-i} e(i) x^*(i-k) = 0 \tag{9.91}$$

for $k = 0, 1, \ldots, p$. Incorporating Eq. (9.90) into Eq. (9.91) yields

$$\sum_{i=0}^{n} \lambda^{n-i} \left\{ d(i) - \sum_{l=0}^{p} w_n(l) x(i-l) \right\} x^*(i-k) = 0$$

Interchanging the order of summation and rearranging terms we have

$$\sum_{l=0}^{p} w_n(l) \left[\sum_{i=0}^{n} \lambda^{n-i} x(i-l) x^*(i-k) \right] = \sum_{i=0}^{n} \lambda^{n-i} d(i) x^*(i-k) \tag{9.92}$$

We may express these equations concisely in matrix form as follows:

$$\boxed{\mathbf{R}_x(n)\mathbf{w}_n = \mathbf{r}_{dx}(n)} \tag{9.93}$$

where $\mathbf{R}_x(n)$ is a $(p+1) \times (p+1)$ exponentially weighted deterministic autocorrelation matrix for $x(n)$

$$\mathbf{R}_x(n) = \sum_{i=0}^{n} \lambda^{n-i} \mathbf{x}^*(i) \mathbf{x}^T(i)$$

with $\mathbf{x}(i)$ the data vector

$$\mathbf{x}(i) = \left[x(i),\ x(i-1),\ \ldots,\ x(i-p) \right]^T$$

and where $\mathbf{r}_{dx}(n)$ is the deterministic cross-correlation between $d(n)$ and $x(n)$,

$$\mathbf{r}_{dx}(n) = \sum_{i=0}^{n} \lambda^{n-i} d(i) \mathbf{x}^*(i) \tag{9.94}$$

Eq. (9.93) is referred to as the *deterministic normal equations*.

Having derived the equations that define the optimum filter coefficients, we may now evaluate the minimum squared error. With

$$\begin{aligned} \mathcal{E}(n) &= \sum_{i=0}^{n} \lambda^{n-i} e(i) e^*(i) = \sum_{i=0}^{n} \lambda^{n-i} e(i) \left\{ d(i) - \sum_{l=0}^{p} w_n(l) x(i-l) \right\}^* \\ &= \sum_{i=0}^{n} \lambda^{n-i} e(i) d^*(i) - \sum_{l=0}^{p} w_n^*(l) \sum_{i=0}^{n} \lambda^{n-i} e(i) x^*(i-l) \end{aligned}$$

note that if $w_n(l)$ are the coefficients that minimize the squared error, then it follows from Eq. (9.91) that the second term is zero and the minimum error is

$$\begin{aligned} \left\{ \mathcal{E}(n) \right\}_{\min} &= \sum_{i=0}^{n} \lambda^{n-i} e(i) d^*(i) \\ &= \sum_{i=0}^{n} \lambda^{n-i} \left\{ d(i) - \sum_{l=0}^{p} w_n(l) x(i-l) \right\} d^*(i) \\ &= \sum_{i=0}^{n} \lambda^{n-i} |d(i)|^2 - \sum_{l=0}^{p} w_n(l) \sum_{i=0}^{n} \lambda^{n-i} x(i-l) d^*(i) \end{aligned}$$

Alternatively, we may express the minimum error in vector form as follows:

$$\boxed{\{\mathcal{E}(n)\}_{\min} = \|\mathbf{d}(n)\|_{\lambda}^2 - \mathbf{r}_{dx}^H(n)\mathbf{w}_n} \tag{9.95}$$

where $\|\mathbf{d}(n)\|_{\lambda}^2$ is the weighted norm of the vector $\mathbf{d}(n) = \left[d(n),\ d(n-1),\ \ldots,\ d(0)\right]^T$.

Since $\mathbf{R}_x(n)$ and $\mathbf{r}_{dx}(n)$ both depend on n, instead of solving the deterministic normal equations directly, for each value of n, we will derive a recursive solution of the form

$$\mathbf{w}_n = \mathbf{w}_{n-1} + \Delta\mathbf{w}_{n-1}$$

where $\Delta\mathbf{w}_{n-1}$ is a correction that is applied to the solution at time $n-1$. Since

$$\mathbf{w}_n = \mathbf{R}_x^{-1}(n)\mathbf{r}_{dx}(n)$$

this recursion will be derived by first expressing $\mathbf{r}_{dx}(n)$ in terms of $\mathbf{r}_{dx}(n-1)$, and then deriving a recursion that allows us to evaluate $\mathbf{R}_x^{-1}(n)$ in terms of $\mathbf{R}_x^{-1}(n-1)$ and the new data vector $\mathbf{x}(n)$. From Eq. (9.94) it follows that the cross-correlation may be updated recursively as follows:

$$\mathbf{r}_{dx}(n) = \lambda\mathbf{r}_{dx}(n-1) + d(n)\mathbf{x}^*(n) \tag{9.96}$$

Similarly, the autocorrelation matrix may be updated from $\mathbf{R}_x(n-1)$ and the new data vector $\mathbf{x}(n)$ using the recursion

$$\mathbf{R}_x(n) = \lambda\mathbf{R}_x(n-1) + \mathbf{x}^*(n)\mathbf{x}^T(n) \tag{9.97}$$

However, since it is the inverse of $\mathbf{R}_x(n)$ that we are interested in, we may apply *Woodbury's Identity*, Eq. (2.29), to Eq. (9.97) to get the desired recursion. Woodbury's identity, repeated below for convenience, is

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^H)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^H\mathbf{A}^{-1}}{1 + \mathbf{v}^H\mathbf{A}^{-1}\mathbf{u}}$$

Note that if we set $\mathbf{A} = \lambda\mathbf{R}_x(n-1)$ and $\mathbf{u} = \mathbf{v} = \mathbf{x}^*(n)$ then we obtain the following recursion for the inverse of $\mathbf{R}_x(n)$:

$$\mathbf{R}_x^{-1}(n) = \lambda^{-1}\mathbf{R}_x^{-1}(n-1) - \frac{\lambda^{-2}\mathbf{R}_x^{-1}(n-1)\mathbf{x}^*(n)\mathbf{x}^T(n)\mathbf{R}_x^{-1}(n-1)}{1 + \lambda^{-1}\mathbf{x}^T(n)\mathbf{R}_x^{-1}(n-1)\mathbf{x}^*(n)} \tag{9.98}$$

To simplify notation, we will let $\mathbf{P}(n)$ denote the inverse of the autocorrelation matrix at time n,

$$\mathbf{P}(n) = \mathbf{R}_x^{-1}(n)$$

and define what is referred to as the *gain vector*, $\mathbf{g}(n)$, as follows:

$$\boxed{\mathbf{g}(n) = \frac{\lambda^{-1}\mathbf{P}(n-1)\mathbf{x}^*(n)}{1 + \lambda^{-1}\mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}^*(n)}} \tag{9.99}$$

Incorporating these definitions into Eq. (9.98) we have

$$\boxed{\mathbf{P}(n) = \lambda^{-1}\Big[\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\Big]} \tag{9.100}$$

An interesting and useful expression for the gain vector may be derived from Eq. (9.99) as follows. Cross-multiplying to eliminate the denominator on the right side of Eq. (9.99), we have

$$\mathbf{g}(n) + \lambda^{-1}\mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}^*(n) = \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}^*(n)$$

Next, bringing the second term on the left to the right side of the equation yields

$$\mathbf{g}(n) = \lambda^{-1}\Big[\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\Big]\mathbf{x}^*(n)$$

Finally, from Eq. (9.100) we see that the term multiplying $\mathbf{x}^*(n)$ is $\mathbf{P}(n)$ and we have

$$\mathbf{g}(n) = \mathbf{P}(n)\mathbf{x}^*(n) \tag{9.101}$$

Thus, the gain vector is the solution to the linear equations

$$\boxed{\mathbf{R}_x(n)\mathbf{g}(n) = \mathbf{x}^*(n)} \tag{9.102}$$

which is the same as the deterministic normal equations for $\mathbf{w}_n$ in Eq. (9.93) except that the cross-correlation vector $\mathbf{r}_{dx}(n)$ has been replaced with the data vector $\mathbf{x}^*(n)$.

To complete the recursion, we must derive the time-update equation for the coefficient vector $\mathbf{w}_n$. With

$$\mathbf{w}_n = \mathbf{P}(n)\mathbf{r}_{dx}(n)$$

it follows from the update for $\mathbf{r}_{dx}(n)$ given in Eq. (9.94) that

$$\mathbf{w}_n = \lambda\mathbf{P}(n)\mathbf{r}_{dx}(n-1) + d(n)\mathbf{P}(n)\mathbf{x}^*(n) \tag{9.103}$$

Next, incorporating the update for $\mathbf{P}(n)$ given in Eq. (9.100) into the first term on the right side of Eq. (9.103), and setting $\mathbf{P}(n)\mathbf{x}^*(n) = \mathbf{g}(n)$ we have

$$\mathbf{w}_n = \Big[\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\Big]\mathbf{r}_{dx}(n-1) + d(n)\mathbf{g}(n)$$

Finally, recognizing that $\mathbf{P}(n-1)\mathbf{r}_{dx}(n-1) = \mathbf{w}_{n-1}$ it follows that

$$\mathbf{w}_n = \mathbf{w}_{n-1} + \mathbf{g}(n)\Big[d(n) - \mathbf{w}_{n-1}^T\mathbf{x}(n)\Big]$$

which may be written as

$$\boxed{\mathbf{w}_n = \mathbf{w}_{n-1} + \alpha(n)\mathbf{g}(n)} \tag{9.104}$$

where

$$\alpha(n) = d(n) - \mathbf{w}_{n-1}^T\mathbf{x}(n) \tag{9.105}$$

is the difference between $d(n)$ and the estimate of $d(n)$ that is formed by applying the *previous* set of filter coefficients, $\mathbf{w}_{n-1}$, to the new data vector, $\mathbf{x}(n)$. This sequence, called the *a priori error*, is the error that would occur if the filter coefficients were not updated. The *a posteriori error*, on the other hand, is the error that occurs after the weight vector is updated,

$$e(n) = d(n) - \mathbf{w}_n^T\mathbf{x}(n)$$

Table 9.6 *Recursive Least Squares*

Parameters:	p = Filter order
	λ = Exponential weighting factor
	δ = Value used to initialize $\mathbf{P}(0)$
Initialization:	$\mathbf{w}_0 = \mathbf{0}$
	$\mathbf{P}(0) = \delta^{-1}\mathbf{I}$
Computation:	For $n = 1, 2, \ldots$ compute
	$\mathbf{z}(n) = \mathbf{P}(n-1)\mathbf{x}^*(n)$
	$\mathbf{g}(n) = \dfrac{1}{\lambda + \mathbf{x}^T(n)\mathbf{z}(n)}\mathbf{z}(n)$
	$\alpha(n) = d(n) - \mathbf{w}_{n-1}^T\mathbf{x}(n)$
	$\mathbf{w}_n = \mathbf{w}_{n-1} + \alpha(n)\mathbf{g}(n)$
	$\mathbf{P}(n) = \dfrac{1}{\lambda}\left[\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{z}^H(n)\right]$

Note that when $\alpha(n)$ is small, the current set of filter coefficients are close to their optimal values (in a least squares sense), and only a small correction needs to be applied to the coefficients. On the other hand, when $\alpha(n)$ is large, the current set of filter coefficients are not performing well in estimating $d(n)$ and a large correction must be applied to update the coefficients.

One final simplification may be realized if we note that, in the evaluation of the gain vector $\mathbf{g}(n)$, and the inverse autocorrelation matrix $\mathbf{P}(n)$, it is necessary to compute the product

$$\boxed{\mathbf{z}(n) = \mathbf{P}(n-1)\mathbf{x}^*(n)} \tag{9.106}$$

Therefore, we may explicitly evaluate this *filtered information vector* and then use it in the calculation of both $\mathbf{g}(n)$ and $\mathbf{P}(n)$. Equations (9.99), (9.100), (9.104), (9.105), and (9.106) form what is known as the exponentially weighted *Recursive Least Squares* (RLS) algorithm, which is summarized in Table 9.6. The special case of $\lambda = 1$ is referred to as the *growing window RLS algorithm.*

The last issue that needs to be addressed concerns the initialization of the RLS algorithm. Since the RLS algorithm involves the recursive updating of the vector $\mathbf{w}_n$ and the inverse autocorrelation matrix $\mathbf{P}(n)$, initial conditions for both of these terms are required. There are two ways that this initialization is typically performed. The first is to build up the autocorrelation matrix recursively according to Eq. (9.97) until it is of full rank (typically $p + 1$ input vectors), and then compute the inverse directly

$$\mathbf{P}(0) = \left[\sum_{i=-p}^{0} \lambda^{-i}\mathbf{x}^*(i)\mathbf{x}^T(i)\right]^{-1}$$

Evaluating the cross-correlation vector $\mathbf{r}_{dx}(0)$ in the same manner,

$$\mathbf{r}_{dx}(0) = \sum_{i=-p}^{0} \lambda^{-i} d(i)\mathbf{x}^*(i)$$

we may then initialize $\mathbf{w}_0$ by setting $\mathbf{w}_0 = \mathbf{P}(0)\mathbf{r}_{dx}(0)$. The advantage of this approach is that optimality is preserved at each step since the RLS algorithm is initialized at $n = 0$ with the vector $\mathbf{w}_0$ that minimizes the weighted least squares error $\mathcal{E}(0)$. A disadvantage, however, is that it requires the direct inversion of $\mathbf{R}_x(0)$, which requires on the order of $(p+1)^3$ operations. In addition, with this approach there will be a delay of $p+1$ samples before any updates are performed. Another approach that may be used is to initialize the autocorrelation matrix as follows:

$$\mathbf{R}_x(0) = \delta\mathbf{I}$$

where δ is a small positive constant, i.e., $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$, and to initialize the weight vector to zero,

$$\mathbf{w}_0 = \mathbf{0}$$

The disadvantage of this approach is that it introduces a bias in the least squares solution [20]. However, with an exponential weighting factor, $\lambda < 1$, this bias goes to zero as n increases. A MATLAB program for the RLS algorithmis given in Fig. 9.24.

Unlike the LMS algorithm, which requires on the order of p multiplications and additions, the RLS algorithm requires on the order of p^2 operations. Specifically, the evaluation of $\mathbf{z}(n)$ requires $(p+1)^2$ multiplications, computing the gain vector $\mathbf{g}(n)$ requires $2(p+1)$ multiplications, finding the a priori error $\alpha(n)$ requires another $p+1$ multiplications, and the update of the inverse autocorrelation matrix $\mathbf{P}(n)$ requires $2(p+1)^2$ multiplications for a total of $3(p+1)^2 + 3(p+1)$. There is a similar number of additions. What is gained with this increase in computational complexity over the LMS algorithm, however, is an increase in performance. Generally, the RLS algorithm converges faster than the LMS algorithm and is less sensitive to eigenvalue disparities in the autocorrelation matrix of $x(n)$ for stationary

The RLS Algorithm

```
function [A,E] = rls(x,d,nord,lambda)
%
   delta=0.001;
   X=convm(x,nord);
   [M,N] = size(X);
   if nargin < 4,   lambda = 1.0;   end
   P=eye(N)/delta;
   W(1,:)=zeros(1,N);
   for k=2:M-nord+1;
       z=P*X(k,:)';
       g=z/(lambda+X(k,:)*z);
       alpha=d(k)-X(k,:)*W(k-1,:).';
       W(k,:)=W(k-1,:)+alpha*g.';
       P=(P-g*z.')/lambda;
   end;
```

Figure 9.24 *A* MATLAB *program for the RLS algorithm.*

processes. On the other hand, without exponential weighting, RLS does not perform very well in tracking nonstationary processes. This is due to the fact that, with $\lambda = 1$, all of the data is equally weighted in estimating the correlations. Although exponential weighting improves the tracking characteristics of RLS, it is not clear how to choose λ and, in some cases, the LMS algorithm may have better tracking properties [6,13] For a more complete discussion of the RLS algorithm, the reader is referred to Reference [20].

Example 9.4.1 *Linear Prediction Using RLS*

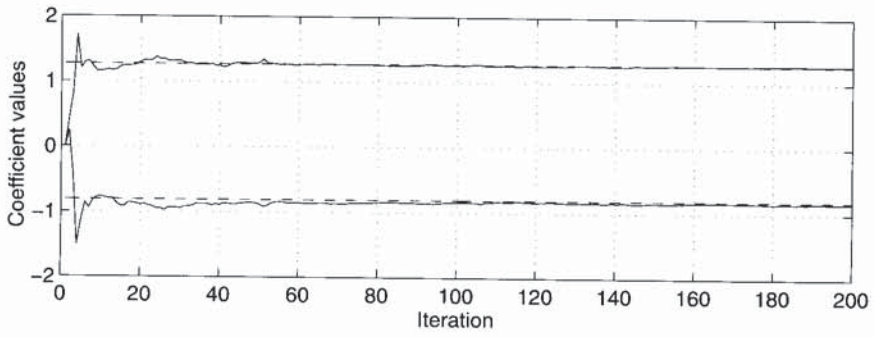
In this example, we compare LMS and RLS for the problem of linear prediction introduced in Example 9.2.1 (p. 509). With $x(n)$ generated according to the difference equation

$$x(n) = 1.2728x(n-1) - 0.81x(n-2) + v(n)$$

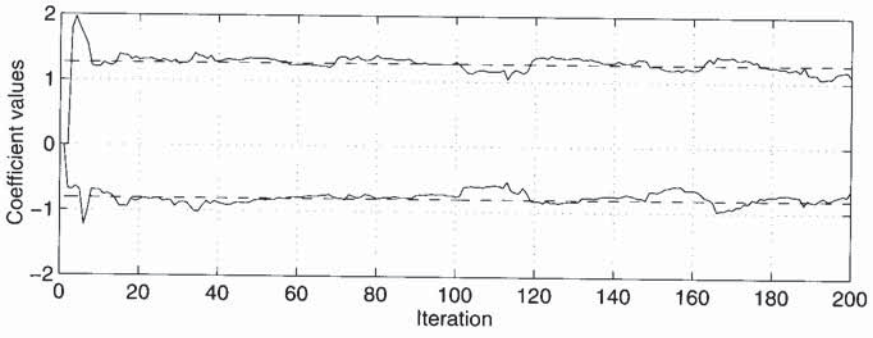
where $v(n)$ is unit variance white noise, we will consider a second-order RLS predictor of the form

$$\hat{x}(n) = w_n(1)x(n-1) + w_n(2)x(n-2)$$

Shown in Fig. 9.25*a* is a plot of the predictor coefficients $w_n(1)$ and $w_n(2)$ versus n using the growing window RLS algorithm ($\lambda = 1$). Compared to the LMS algorithm in Fig. 9.10,



(a)



(b)

Figure 9.25 *Linear prediction of a second-order autoregressive process using recursive least squares. The dashed lines indicate the correct values for the coefficients. (a) Growing window RLS. (b) Exponentially weighted RLS with* $\lambda = 0.95$.

what is clearly evident is that the RLS algorithm has a much higher rate of convergence. Fig. 9.25*b*, on the other hand, shows the predictor coefficients using the exponential window RLS algorithm with $\lambda = 0.95$. What we observe is an increase in the fluctuations of the weights with the exponential window. This is due to the decrease in the effective length of the data window that is used to evaluate $\mathbf{R}_x(n)$. With a growing window RLS algorithm, if the signal properties are not changing in time then $\mathbf{R}_x(n)$ and the weights $\mathbf{w}_n$ become more stable as n increases.

As a final comparison, let us evaluate what happens if $x(n)$ is generated according to the time-varying difference equation

$$x(n) = a_n(1)x(n-1) - 0.81x(n-2) + v(n)$$

where $v(n)$ is unit variance white noise and $a_n(1)$ is a time-varying coefficient of the following form

$$a_n(1) = \begin{cases} 1.2728 & ; \quad 0 \leq n < 100 \\ 0 & ; \quad 100 \leq n \leq 200 \end{cases}$$

This corresponds to a filter having a pair of poles at radius $r = 0.9$ and angle $\omega_0 = \pi/4$ when $a_n(1) = 1.2728$ and radius $r = 0.9$ and angle $\omega_0 = \pi/2$ when $a_n(1) = 0$. Thus, at time $n = 100$ the poles move from $\omega_0 = \pi/4$ to $\omega_0 = \pi/2$. Shown in Fig. 9.26*a* is a plot of $w_n(1)$ versus n for $\lambda = 1$. What we see is that the growing window RLS algorithm is not able to track the time-variations effectively due to the infinite length window (infinite memory). Shown in Fig. 9.26*b*, on the other hand, is a plot of $w_n(1)$ versus n for $\lambda = 0.9$. Note that with this decrease in the weighting factor, the RLS algorithm is able to more accurately track the true values. Finally, shown in Fig. 9.26*c* is a plot of the coefficients obtained with the LMS algorithm using a step size $\mu = 0.02$. As we see, the performance of the LMS algorithm in tracking nonstationary processes is similar to the exponentially weighted RLS algorithm.

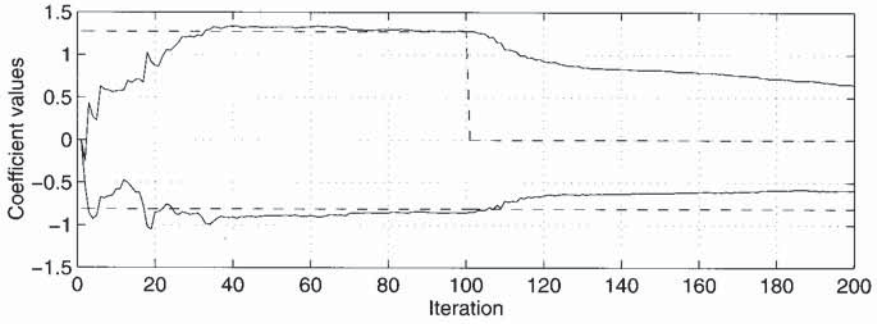
9.4.2 Sliding Window RLS

The RLS algorithm minimizes the exponentially weighted least squares error $\mathcal{E}(n)$ given in Eq. (9.89). With a growing window RLS algorithm ($\lambda = 1$), each of the squared errors $|e(i)|^2$ from $i = 0$ to $i = n$ are equally weighted, whereas with an exponentially weighted RLS algorithm ($\lambda < 1$), the squared errors $|e(i)|^2$ become less important for values of i that are small compared to n. In both cases, however, the RLS algorithm has infinite memory in the sense that all of the data beginning from time $n = 0$ will affect the values of the coefficients $\mathbf{w}_n$. In some applications this may not be desirable. For example, for a nonstationary process whose statistics are changing rapidly in time, a small value for λ will be necessary in order to track the process. An alternative, however, is to minimize the sum of the squares of $e(i)$ over a finite window,[8]

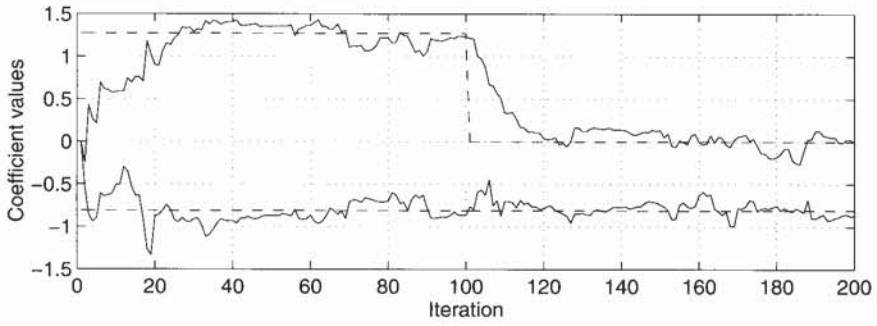
$$\mathcal{E}_L(n) = \sum_{i=n-L}^{n} |e(i)|^2 \tag{9.107}$$

In addition to being able to more easily track nonstationary processes, the finite window allows for any data *outliers* to be *forgotten* after a finite number of iterations.

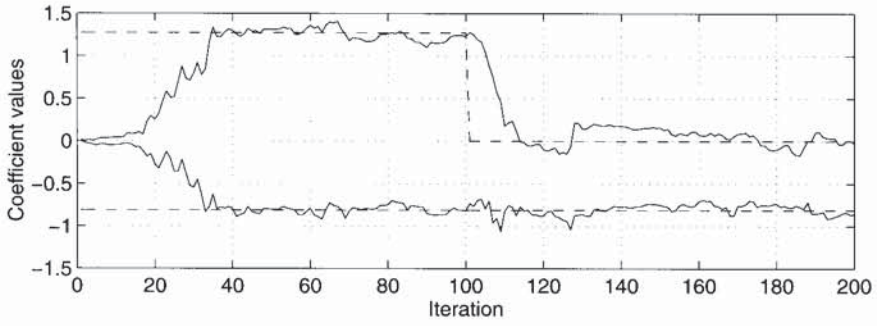
[8]Note that in the special case of $L = 0$, a gradient descent approach to minimize $\mathcal{E}_0(n)$ results in the LMS algorithm.



(a)



(b)



(c)

Figure 9.26 *Linear prediction of a nonstationary second-order autoregressive random process. The dashed lines indicate the correct values for the coefficients. (a) Growing window RLS. (b) Exponentially weighted RLS with* $\lambda = 0.9$. *(c) LMS algorithm with a step size* $\mu = 0.02$.

Following the same steps used to derive the deterministic normal equations for minimizing the exponentially weighted squared error, the filter coefficients $\mathbf{w}_n$ that minimize $\mathcal{E}_L(n)$ are found to be solutions to the equations

$$\mathbf{R}_x(n)\mathbf{w}_n = \mathbf{r}_{dx}(n) \tag{9.108}$$

where

$$\mathbf{R}_x(n) = \sum_{i=n-L}^{n} \mathbf{x}^*(i)\mathbf{x}^T(i) \tag{9.109}$$

and

$$\mathbf{r}_{dx}(n) = \sum_{i=n-L}^{n} d(i)\mathbf{x}^*(i)$$

Using a procedure similar to that used to derive the growing window RLS algorithm, a *sliding window* RLS algorithm may be developed that solves Eq. (9.108) recursively with a computational complexity on the order of p^2 operations. The sliding window RLS algorithm consists of the following two steps.

1. Given the solution $\mathbf{w}_{n-1}$ to Eq. (9.108) at time $n-1$, with the new data value $x(n)$, the weight vector $\widetilde{\mathbf{w}}_n$ is found that minimizes the error

$$\mathcal{E}_{L+1}(n) = \sum_{i=n-L-1}^{n} |e(i)|^2 \tag{9.110}$$

2. The weight vector $\mathbf{w}_n$ that minimizes $\mathcal{E}_L(n)$ is then determined by discarding the last data point, $x(n-L-1)$.

In the first step, note that we find the vector $\widetilde{\mathbf{w}}_n$ that minimizes the error $\mathcal{E}_{L+1}(n)$ that results from the addition of one additional data value. Therefore, we may use the growing window RLS algorithm to find $\widetilde{\mathbf{w}}_n$ from $\mathbf{w}_{n-1}$ as follows:

$$\begin{aligned}
\mathbf{g}(n+1) &= \frac{1}{1+\mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}(n)}\mathbf{P}(n-1)\mathbf{x}^*(n) \\
\widetilde{\mathbf{w}}_n &= \mathbf{w}_{n-1} + \mathbf{g}(n)[d(n) - \mathbf{w}_{n-1}\mathbf{x}^*(n)] \\
\widetilde{\mathbf{P}}(n) &= \mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)
\end{aligned}$$

Note that $\widetilde{\mathbf{P}}(n)$ is the inverse of the matrix

$$\widetilde{\mathbf{R}}_x(n) = \sum_{k=n-L-1}^{n} \mathbf{x}^*(k)\mathbf{x}^T(k)$$

which is based on $L+2$ data values, and that $\widetilde{\mathbf{w}}_n$ is the solution to

$$\widetilde{\mathbf{R}}_x(n)\widetilde{\mathbf{w}}_n = \widetilde{\mathbf{r}}_{dx}(n)$$

where

$$\widetilde{\mathbf{r}}_{dx}(n) = \sum_{k=n-L-1}^{n+1} d(k)\mathbf{x}^*(k)$$

In the second step of the recursion, the last data point $x(n-L-1)$ is discarded to restore the $(L+1)$ point window. This is accomplished with a slight modification to the growing window RLS algorithm. Specifically, we begin with the matrix update

$$\mathbf{R}(n) = \widetilde{\mathbf{R}}(n) - \mathbf{x}^*(n-L)\mathbf{x}^T(n-L-1)$$

and the update of $\mathbf{r}_{dx}(n)$

$$\mathbf{r}_{dx}(n) = \widetilde{\mathbf{r}}_{dx}(n) - d(n-L-1)\mathbf{x}^*(n-L-1)$$

Finally, using the matrix inversion lemma and retracing the steps used to derive the RLS

algorithm we obtain the following update equations

$$\begin{aligned}\widetilde{\mathbf{g}}(n+1) &= \frac{1}{1-\mathbf{x}^T(n-L-1)\widetilde{\mathbf{P}}(n)\mathbf{x}^*(n-L-1)}\widetilde{\mathbf{P}}(n)\mathbf{x}^*(n-L-1)\\ \mathbf{w}_n &= \widetilde{\mathbf{w}}_n - \widetilde{\mathbf{g}}(n)[d(n-L-1) - \widetilde{\mathbf{w}}_n\mathbf{x}^*(n-L-1)]\\ \mathbf{P}(n) &= \widetilde{\mathbf{P}}(n) + \widetilde{\mathbf{g}}(n)\mathbf{x}^T(n-L-1)\widetilde{\mathbf{P}}(n)\end{aligned}$$

which yields the desired solution and completes the derivation of the sliding window RLS algorithm. Compared to exponentially weighted RLS, the sliding window RLS requires about twice the number of multiplications and additions and, in addition, requires that $p + L$ values of $x(n)$ be stored. This storage requirement may potentially be a problem for long windows.

9.4.3 Summary

In this chapter, we have looked at a number of techniques for processing nonstationary signals using adaptive filters. These techniques have been extensively used in a variety of applications including system identification, signal modeling, spectrum estimation, noise cancellation, and adaptive equalization. Our treatment of adaptive filtering began with the design of an FIR filter to minimize the mean-square error $\xi(n) = E\{|e(n)|^2\}$ between a desired process $d(n)$ and an estimate of this process that is formed by filtering another process $x(n)$. First, we considered a steepest descent approach to minimize $\xi(n)$. However, since the gradient of $\xi(n)$ involves an expectation of $e(n)\mathbf{x}^*(n)$, this approach requires knowledge of the statistics of $x(n)$ and $d(n)$ and, therefore, is of limited use in practice. Next, we replaced the ensemble average $E\{|e(n)|^2\}$ with the instantaneous squared error $|e(n)|^2$. This led to the LMS algorithm, a simple and often effective algorithm that does not require any ensemble averages to be known. For wide-sense stationary processes, the LMS algorithm converges in the mean if the step size is positive and no larger than $2/\lambda_{\max}$, and it converges in the mean-square if the step size is positive and no larger than $2/\mathrm{tr}(\mathbf{R}_x)$. We then looked at several modifications of the LMS algorithm. The first was the normalized LMS algorithm, which simplifies the selection of the step size to ensure that the coefficients converge. Next was the leaky LMS algorithm, which is useful in overcoming the problems that occur when the autocorrelation matrix of the input process is singular. We then looked at the block LMS algorithm and the sign algorithms, which are designed to increase the efficiency of the LMS algorithm. In the block LMS algorithm, the filter coefficients are held constant over blocks of length L, which allows for the use of fast convolution algorithms to compute the filter output. The sign algorithms, on the other hand, achieve their simplicity by replacing $e(n)$ with $\mathrm{sgn}\{e(n)\}$, or $\mathbf{x}(n)$ with $\mathrm{sgn}\{\mathbf{x}(n)\}$, or both. Finally, we discussed how it would be beneficial to consider using a variable step size in the LMS algorithm. Ideally, when the filter coefficients are far from their optimum values, the step size should be large. Then, as the cofficients begin to converge to the optimum solution, the step size should be small in order to reduce the excess mean-square error. It is difficult, however, to define a set of rules for adapting the step size so that the adaptive filter will have a small excess mean-square error while, at the same time, maintaining the ability of the filter to respond quickly to changes in the signal statistics. Finally, we looked at the lattice filter as an alternative structure to use as an adaptive filter. Due to the orthogonalization of the input process, the gradient adaptive lattice filter converges more rapidly than the LMS adaptive